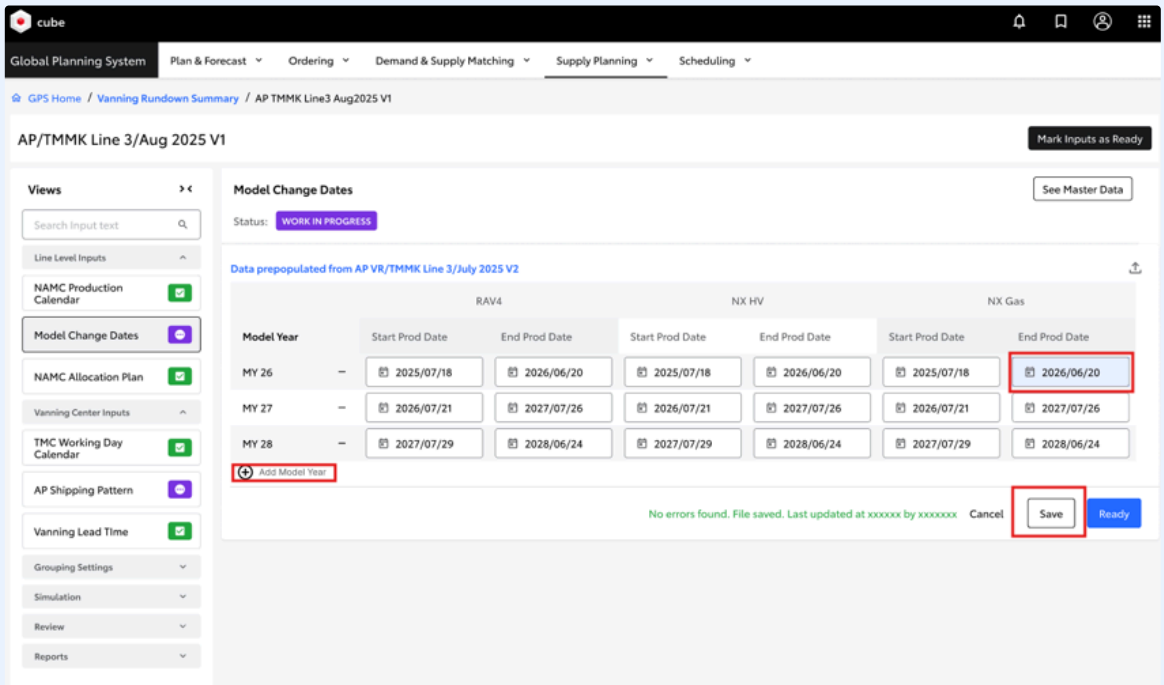
SPPL-192: Model Change Dates POST API technical workflow

Jira Ticket: SPPL-192: BE | Create technical workflow for Model Change Dates Get & Post API on parameter inputs page including DB details ACCEPTED

This page provides comprehensive information about the POST API for Model Change Dates on the Input params page, to update model change dates data.



Supply Planning POST Model Change Dates API Details:

1. **Hosted URL**: https://api.spln.cube<env>.toyota.com/<env>
2. **API Name:** <HOSTED_URL>/v/1/1/sp/model-change-dates
3. **Path Param**: N/A
4. **Query Param**: N/A
5. **Method:** POST
6. **Authorization**: Bearer token (**required**)
7. **Lambda Name**: spln-<env>-update-model-change-dates-v1-1
8. **API Request Body**:

```
{
  "scenarioId": "Getsudo/TMMI/Line1_Cycle_V1",
  "userEmail": "user@toyota.com",
  "data": [
    {
      "modelYear": "MY 25",
      "subSeries": "nxHV",
      "startProdDate": "2025-01-01",
```

```
      "endProdDate": "2025-12-30"
    },
    {
      "modelYear": "MY 25",
      "subSeries": "RAV4",
      "startProdDate": "2025-01-01",
      "endProdDate": "2025-12-30"
    }
  ]
}
```

9. **API Response:**
   - **Success response with status 200:**

```
{
  "message": "Successfully updated data."
}
```

   - **Error Response:**
     - **any internal server error occurred with status 500:**

```
{
  "errorMessage": "Internal Server Error"
}
```

     - **any unauthorized error occurred with status 401 (Managed by the AWS API Gateway integrated Custom Authorizer):**

```
{
  "message": "Unauthorized"
}
```

     - **If request is in-valid with status 400:**

```
{
  "errorMessage": [<"ValidationError: validation error message”>]
}
```

     - **List of possible validation error message:**
       1. "ValidationError: scenarioId is required and must be a uuid."
       2. "ValidationError: userEmail is required and must be a string."
       3. "ValidationError: Scenario doesn't exist."
       4. "ValidationError: Invalid userEmail."
       5. “ValidationError: data is required and must be an array with atleast 1 item.”
       6. “ValidationError: modelYear is required and must be a string in the format MY YY.”
       7. "ValidationError: subSeries is required and must be a string."
       8. “ValidationError: startProdDate is required and must be a string in the format YYYY-MM-DD.”

9. “ValidationError: endProdDate is required and must be a string in the format YYYY-MM-DD.”
10. “ValidationError: For subseries ${subSeries}, model year ${modelYear}: startProdDate must be earlier than endProdDate.”
11. “ValidationError: For subseries ${subSeries}, model year ${modelYear}: endProdDate must be later than startProdDate.”
12. “ValidationError: For subseries ${subSeries}, model year ${modelYear}: startProdDate must be one day after previous model year endProdDate.”
13. “ValidationError: For subseries ${subSeries}: First model year start must be same or earlier than scenario start.”
14. “ValidationError: For subseries ${subSeries}: Last model year start must be same or later than scenario end.”

10. **Table:**
    a. **scenarios**
    b. **model_change_date**

SP Update Model Change Dates API Workflow:

- **Technical workflow:**
  - Description:
    i. The Supply Planning user logs in to the application.
    ii. After the user provides creates a scenario, opens it, clicks on Model Change Dates and updates a field, a request is sent to the SP Post Model Change Dates endpoint: ***sp/model-change-dates*** with request body:

```json
{
  "scenarioId": "Getsudo/TMMI/Line1_Cycle_V1",
  "userEmail": "user@toyota.com",
  "data": [
    {
      "modelYear": "MY 25",
      "subSeries": "nxHV",
      "startProdDate": "2025-01-01",
      "endProdDate": "2025-12-30"
    },
    {
      "modelYear": "MY 25",
      "subSeries": "RAV4",
      "startProdDate": "2025-01-01",
      "endProdDate": "2025-12-30"
    }
  ]
}
```

iii. After successful authentication, the **spln-update-model-change-dates-v1-1** lambda is triggered by the SP update scenario Model Change Dates data endpoint: ***sp/*model-change-dates**. If authentication fails, an error with status 401 (*refer: error response-**any unauthorized error occurred with status 401**)* will be returned and sent to client.

iv. After user authentication has been confirmed, the **spln-update-model-change-dates-v1-1** lambda will perform validation on the request payload. If payload is valid will proceed to next step else throw validation error (r*efer: error response**: Section if request is in-valid with status 400 and List of possible validation error message***). Below query is used to check if provided scenario is valid:

```
select * from supply_planning.scenarios where scenario_id = :scenarioId and
is_active = true;
```

v. Fetch the existing data and then perform validations ref(vi):

```
select * from supply_planning.model_change_date where scenario_id =
:scenarioId;
```

vi. Also, validate the start and end dates by below rules:

1. The start date must be **earlier than** the end date for a MY (for a specific subseries)
2. The end date must be **later than** the start date for a MY (for a specific subseries)
3. The start date for next MY (same subseries) must be exactly +1 day from previous MY end date (same subseries)
4. To be able to mark input as ready:
   - Start date for 1st MY for all subseries should be **same month or earlier than** scenario start date (timeframe in landing page - There is no restriction on how early the date can be)
   - End date for last MY for all subseries should be **same month or later than** scenario end date (timeframe in landing page- There is no restriction on how later the date can be)

   Example:

   a. Scenario Timeframe: June, 2024 to Sep, 2027

   b. Expected Model Change Dates Data -

Subseries	Model Year	Start Date	End Date
A	MY 24	2024-06-01	2025-05-31
A	MY 25	2025-06-01	2026-05-31
A	MY 26	2026-06-01	2027-09-30
B	MY 24	2024-06-01	2025-05-31
B	MY 25	2025-06-01	2026-05-31
B	MY 26	2026-06-01	2027-09-30
C	MY 24	2024-06-01	2025-05-31
C	MY 25	2025-06-01	2026-05-31
C	MY 26	2026-06-01	2027-09-30

vii. After the request payload is successfully validated, the **sp-update-model-change-dates-v1-1** lambda execute below query to update scenario Model Change Dates data in the **model_change_date** table.

```
await prisma.$executeRaw`
INSERT INTO supply_planning.model_change_date (
  scenario_id,
  model_year,
  sub_series_description,
  model_year_start_date,
  model_year_end_date,
  created_by,
  updated_by,
  last_updated
)
SELECT
  ${scenarioId}::uuid AS scenario_id,
  v.model_year,
  v.sub_series_description,
  v.model_year_start_date,
  v.model_year_end_date,
  ${userEmail}::text AS created_by,
  ${userEmail}::text AS updated_by,
  NOW() AS last_updated
FROM (
  VALUES
    ${Prisma.join(
      input.map(item => Prisma.sql`(
        ${item.modelYear}::text,
        ${item.subSeriesDescription}::text,
        ${item.modelYearStartDate}::date,
        ${item.modelYearEndDate}::date
      )`)
    )}
) AS v(model_year, sub_series_description, model_year_start_date, model_year_end_date)
ON CONFLICT (scenario_id, model_year, sub_series_description)
DO UPDATE SET
  model_year_start_date = EXCLUDED.model_year_start_date,
  model_year_end_date   = EXCLUDED.model_year_end_date,
  updated_by            = ${userEmail}::text,
  last_updated          = NOW()
`;
```

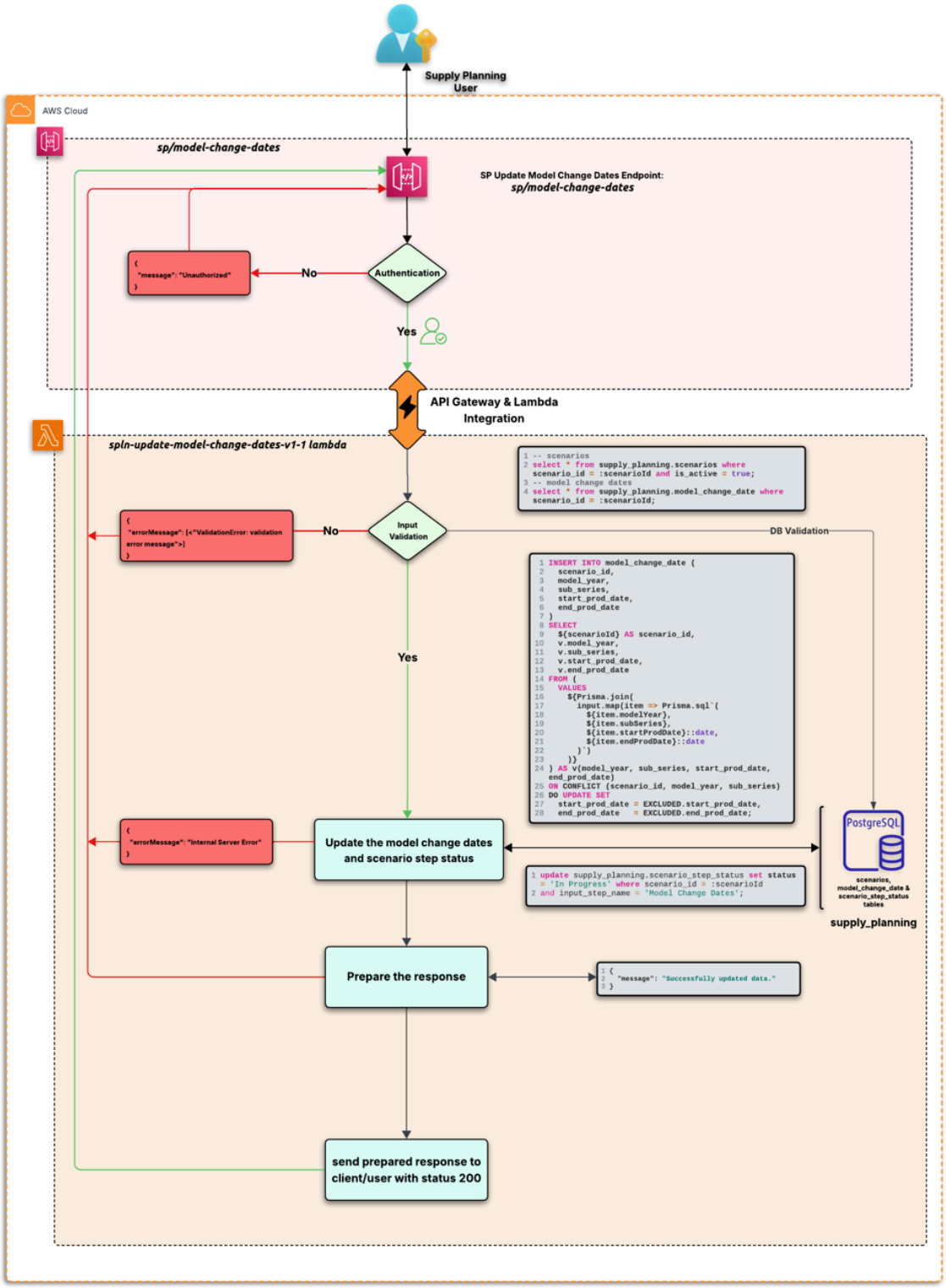
viii. Update the step status to “In Progress”

```
insert into supply_planning.scenario_step_status (scenario_id, input_step_type, input_step_name, status, updated_by, last_updated)
values (${scenarioId}::uuid, ${stepType}, ${stepName}, ${status}, ${userEmail}, now())
on conflict (scenario_id, input_step_name)
do update set status = ${status}, updated_by = ${userEmail}, last_updated = now()
returning scenario_step_status_id::uuid;
```

ix. After retrieving the Model Change Dates data, format and return the below response to the client with a status code of 200:

```
{
  "message": "Successfully updated data."
}
```

x. If there is an error while updating the data, an “Internal Server Error” with a status code of 500(*refer: error response-**any internal server error occurred with status 500***) will be returned.



SP Model Change Dates Post API Techncial workflow

Queries: